Graphs

Objectives:

- Describe a graph
- Describe how graph can be represented in memory
- Implements the various operations on graphs
- Describe applications of graphs

Introduction

- Graph is another important non linear data structure. This data structure is used to represent relationship between pairs of elements, which are not necessarily hierarchical in nature.
- A graph is defined as:

"Graph G is a ordered set (V,E), where V(G) represent the set of elements, called vertices, and E(G) represents the edges between these vertices."

- Graphs can be
  - Undirected
  - Directed

Graphs

Figure shows a sample graph

V(G)={v1,v2,v3,v4,v5}

E(G)={e1,e2,e3,e4,e5}

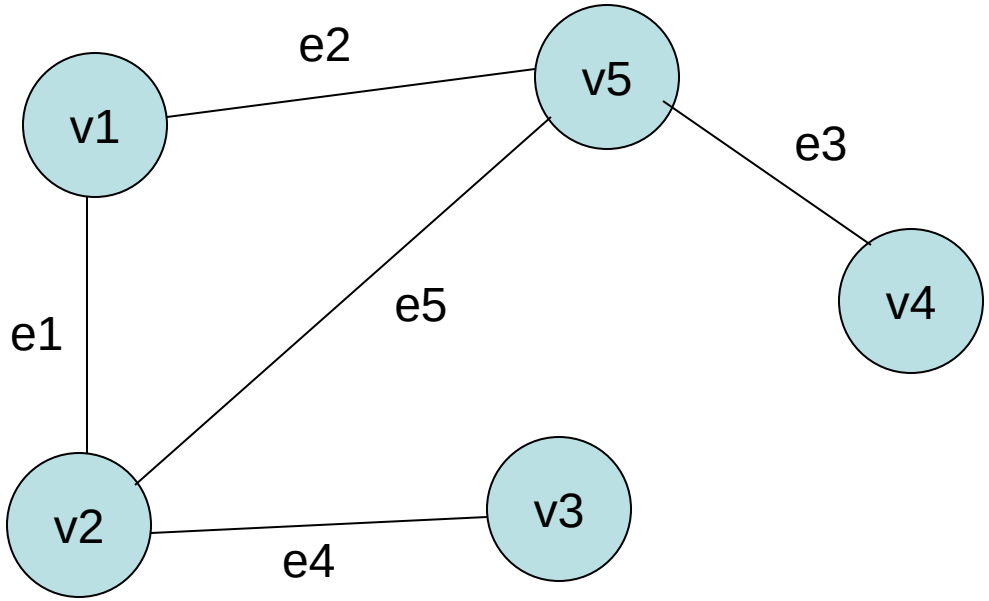


Fig . (a) Undirected Graph

Graph

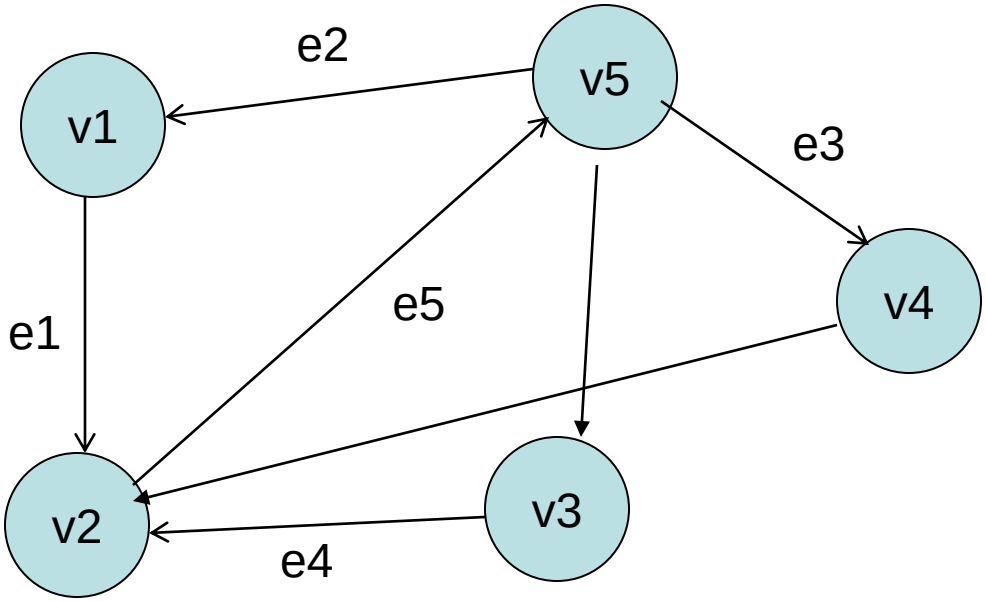


Fig. (b) Directed Graph

In directed graph, an edge is represented by an ordered pair (u,v) (i.e.=(u,v)), that can be traversed only from u toward v.

Graph Terminology

- **Adjacent Vertices:**

As an edge e is represented by pairs of vertices denoted by [u,v]. The vertices u and v are called endpoints of e. these vertices are also called adjacent vertices or neighbors.

- **Degree of a vertex**:

The degree of vertex u, written as deg(u), is the number of edges containing u. If deg(u)=0, this means that vertex u does not belong to any edge, then vertex u is called an isolated vertex.

Graph Terminology

- **Path:**

A path P of length n from a vertex u to vertex v is defined as sequence of (n+1) vertices i.e.

$$P=(v1,v2,v3,\ldots\ldots vn+1)$$

Such that u=v1, v=vn+1

- The path is said to be closed if the endpoints of the path are same i.e. v1=vn+1.
- The path is said to be simple if all the vertices in te sequence are distinct, with the exception that v1=vn+1.In that case it is known as closed simple path.

Graph Terminology

- **Cycle:**

A cycle is closed simple path with length two or more. Sometimes, a cycle of length k (i.e. k distinct vertices in the path) is known as k-cycle.

- **Connected Graph:**

A graph is said to be connected if there is path between any two of its vertices, i.e. there is no isolated vertex.

A connected graph without any cycles is called a tree. Thus we can say that tree is a special graph.

Graph Terminology

- **Complete Graph:**

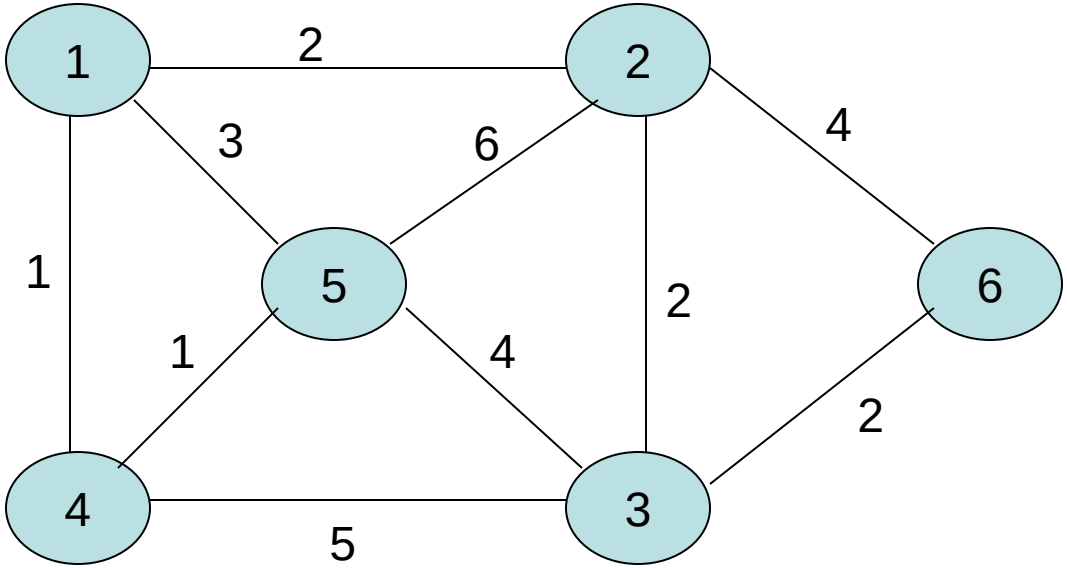
A graph G is said to be complete or fully connected if there is a path from every vertex to every other vertex. A complete graph with n vertices will have n(n-1)/2 edges.

Graph Terminology

- **Weighted Graph:**

A graph is said to be weighted graph if every edge in the graph is assigned some data. The weight is denoted by w(e).

w(e) is non negative value that may be representing the cost of moving along that edge or distance between the vertices.



Weighted undirected graph

Graph Terminology

- **Multiple Edges:**

Distinct edges e and e’ are called multiple edges if they connect the same end points i.e., if e=[u,v] and e’=[u,v]

Multigraph:

A graph containing multiple edges.

Loop:

An edge is a loop if it has identical endpoints, i.e. if e=[u,u]

Terms relevance with directed graph only

- **Directed Graph:**

A directed graph G is graph in which each edge is assigned a certain direction, i.e. each edge is ordered pair of (u,v) of vertices rather than an unordered pair [u,v]

Suppose G is a directed graph with e=(u,v) as one of the edge, then

- e begins at u and ends at v.
- u is the origin of e, and v is the destination of e
- u is predecessor of v, and v is the successor of u.

Directed Graph terminology

- **Out degree and In degree of a vertex:**

The out-degree of vertex u, denoted by outdeg(u), is the number of edges originating at u.

The in-degree of a vertex u, denoted by indeg(u), is the number of edges terminating at u.

Directed Graph terminology

- **Source and sink:**

A vertex u is called a source if it has a out-degree greater than zero, but zero in-degree.

A vertex u is called a sink if it has a in-degree greater than zero, but zero out-degree.

Directed Graph terminology

- **Reachability:**

Vertex v is said to be reachable from u if there exists a path from vertex u to vertex v.

- **Strongly Connected:**

A directed graph G is said to be strongly connected, if for each pair u, v of the vertices in G, if there exists a path from u to v, there must exist path from v to u; otherwise the directed graph is unilaterally connected.

Directed Graph terminology

- **Parallel Edges:**

Distinct edges e and e’ are called parallel edges if they connect the same source and terminal vertices, i.e.if e=(u,v) and e’=(u,v)

- **Simple Directed Graph:**

A directed graph G is said to be simple, if there are no parallel edges. A simple directed graph may have loops, but it can not have more than one loop a a given vertex.

- **Directed cyclic Graph:**

A directed cyclic graph G is a graph without cycle(s).

Representation of Graph

- **Using an adjacency matrix**
- **Using an adjacency list**

Adjacency Matrix Representation

Consider a directed graph G=(V,E). We will assume that the vertices are numbered 1,2,3,..|v|, in some arbitrary manner. The adjacency matrix representation of a graph G then consists of |v| X |v| matrix A=(aij), such that

$$aij = \begin{cases} 1 & \text{if } (I,j) € E \\ 0 & \text{otherwise} \end{cases}$$

Adjacency Matrix Representation

For undirected graph G=(V,E), the adjacency matrix representation is also consists of |v|X|v| matrix A=(aij) but its elements are as follows:

$$aij = \begin{cases} 1 & \text{if either } [I,j] \text{ € } E \text{ or } [j,i] \text{ € } E \\ 0 & \text{otherwise} \end{cases}$$

Adjacency Matrix Representation

$$
\begin{array}{c c}
 & \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\
\begin{array}{c} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} &
\begin{pmatrix}
0 & 1 & 0 & 0 & 1 \\
1 & 0 & 1 & 0 & 1 \\
0 & 1 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 \\
1 & 1 & 0 & 1 & 0
\end{pmatrix}
\end{array}
$$

Adjacency Matrix Representation of undirected graph in fig (a)

Adjacency Matrix Representation

$$
\begin{array}{c c}
 & \begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \end{array} \\
\begin{array}{c} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} &
\begin{pmatrix}
0 & 1 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 \\
0 & 1 & 0 & 0 & 0 \\
0 & 1 & 0 & 0 & 0 \\
1 & 0 & 1 & 1 & 0
\end{pmatrix}
\end{array}
$$

Adjacency Matrix Representation of directed graph in fig (b)

Adjacency Matrix Representation

- Adjacency matrix for non weighted graphs that contains entries of only 0 and 1 is called bit matrix or a Boolean matrix.
- Adjacency matrix representation of a graph requires $0(v^2)$ memory location irrespective of their number of edges in the graph.

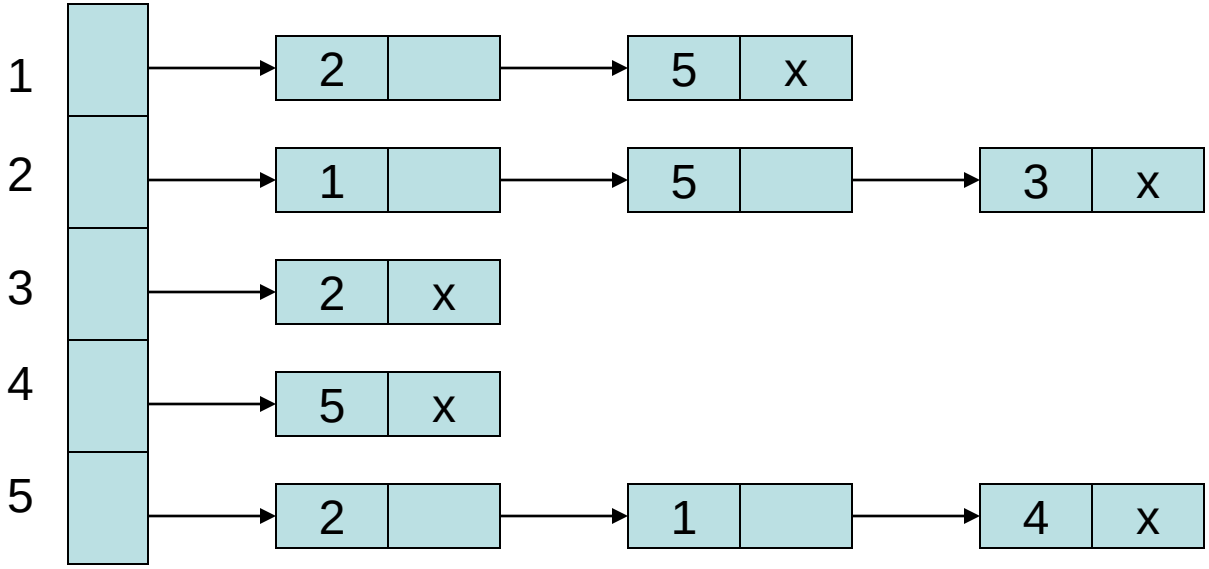
Adjacency List Representation

- The Adjacency List Representation of a graph G=(V,E) consists of an array Adj of |V| lists, one for each vertex in V.
- For each u € V , the adjacency list Adj[u] contains all the vertices v such that there is an edge (u,v) € E i.e. Adj[u] consists of all the vertices adjacent to u in G.
- The vertices in each adjacency list are stored in an arbitrary order.

Adjacency List Representation

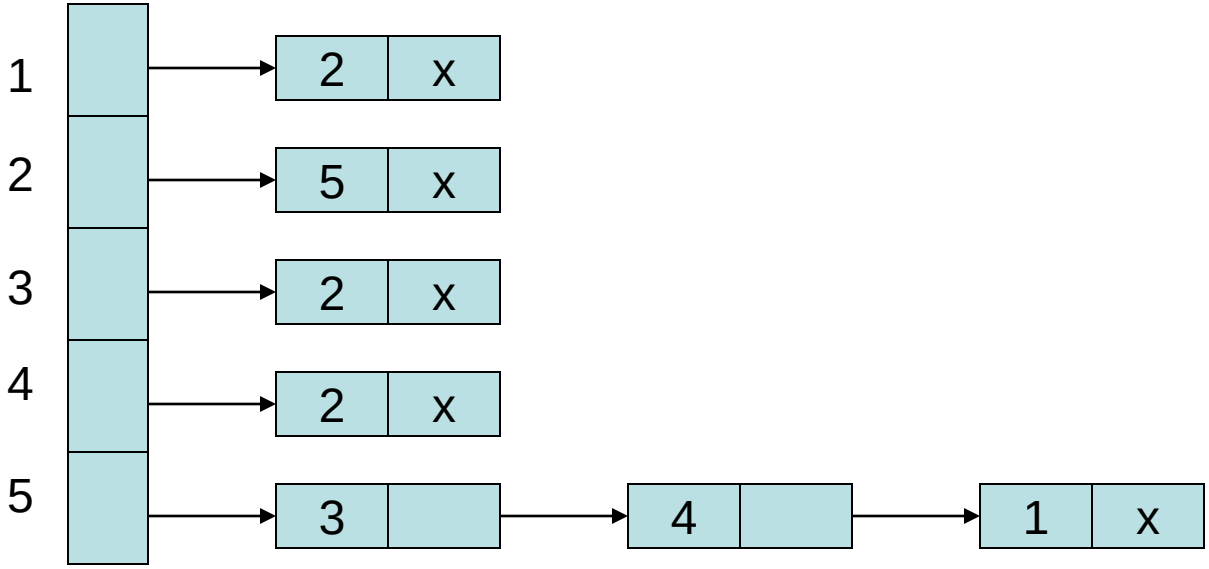
- If G is an undirected graph, the sum of the lengths of all the adjacency lists 2|E|.
- If G is an directed graph, the sum of the lengths of all the adjacency lists |E|.
- Although the adjacency list representation requires very less memory as compared to the adjacency matrix.
- The simplicity adjacency matrix make it preferable when graphs are reasonably small.

Adjacency List Representation



Adjacency list for undirected graph of fig.(a)

Adjacency List Representation



Adjacency list for undirected graph of fig.(b)

Implementation of Adjacency list in C

- **For non weighted graph:**

```
#define MAX 50
Typedef struct nodetype
{
    int vertex;
    struct nodetype *next;
}graphnode;
graphnode *adj[MAX];
```

- **For weighted graph:**

```
Typedef struct nodetype1
{
    int vertex;
    int weight;
    struct nodetype1 *next;
}graphnode1;
graphnode1 *adj[MAX];
```

Operations on Graphs

- **Creating an empty graph:**

To create an empty graph, the entire adjacency list is set to NULL.

```
Void creatgraph(graphnode *adj[], int num)
{
    int i;
    for(i=1;i<=num;i++)
        adj[i]=NULL;
}
Void creatweightedgraph(graphnode1 *adj[], int num)
{
    int i;
    for(i=1;i<=num;i++)
        adj[i]=NULL;
}
```

- **Entering Graph information:**

The graph information is entered as shown…

```
Void inputgraph(graphnode *adj[], int num)
{
    graphnode *ptr,*last;
    int I,j,m,val;
     for(i=1;i<=num;i++)
    {
            last=NULL;
            printf("\no. of nodes in the adjacency list of node  %d",i);
            scanf("%d",&m);
            for(j=1;j<=m;j++)
            {
                    printf("enter node%d:",j);
                    scanf("%d",&val);
                    ptr=(graphnode*)malloc(sizeof(graphnode));
                    ptr->vertex=val;
                    ptr->next=NULL;
                    if(adj[i]==NULL)
                            adj[i]=last=ptr;
                    else  {
                            last->next=ptr;
                            last=ptr;
                          }
            }
    }
}
```

- **Entering Graph information:**

The graph information is entered as shown…

```
Void inputweightedgraph(graphnode1 *adj[], int num)
{
	graphnode1 *ptr,*last;
	int I,j,m,val,wt;
	 for(i=1;i<=num;i++)
	{
		last=NULL;
		printf("\no. of nodes in the adjacency list of node  %d",i);
		scanf("%d",&m);
		for(j=1;j<=m;j++)
		{
			printf("enter node%d:",j);
			scanf("%d",&val);
			printf("enter weight for edge %d%d:",I,val);
			scanf("%d",&wt);
			ptr=(graphnode*)malloc(sizeof(graphnode));
			ptr->vertex=val;
			ptr->weight=wt;
			ptr->next=NULL;
			if(adj[i]==NULL)
				adj[i]=last=ptr;
			else  {
				last->next=ptr;
				last=ptr;
			}
		}
	}
}
```

Operations on Graphs

- **Outputting a Graph:**

```
Void printgraph(graphnode *adj[],int num)
{
    graphnode *ptr;
    int I;
    for(i=1;i<=num;i++)
    {
        ptr=adj[i];
        printf("%d",i);
        while(ptr!=NULL)
        {
            printf("->%d",ptr->vertex);
            ptr=ptr->next;
        }
        printf("\n");
    }
}
```

Operations on Graphs

- **Outputting a Graph:**

```
Void printweightedgraph(graphnode1 *adj[], int num)
{
    graphnode1 *ptr;
    int I;
    for(i=1;i<=num;i++)
    {
        ptr=adj[i];
        printf("%d",i);
        while(ptr!=NULL)
        {
            printf("->%d,%d",ptr->vertex,ptr->weight);
            ptr=ptr->next;
        }
        printf("\n");
    }
}
```

Operations on Graphs

- **Deleting a Graph:**

```
Void deletegraph(graphnode *adj[], int n)
{
    int I;
    graphnaode *temp,*ptr;
    for(i=1;i<=n;i++)
    {
        ptr=adj[i];
        while(ptr!=NULL)
        {
            temp=ptr;
            ptr=ptr->next;
            free(temp);
        }
        adj[i]=NULL;
    }
}
```

Operations on Graphs

- **Deleting a Graph:**

```
Void deleteweightedgraph(graphnode1 *adj[], int n)
{
    int I;
    graphnaode1 *temp,*ptr;
    for(i=1;i<=n;i++)
    {
        ptr=adj[i];
        while(ptr!=NULL)
        {
            temp=ptr;
            ptr=ptr->next;
            free(temp);
        }
        adj[i]=NULL;
    }
}
```

Traversal

Many applications of the graphs requires examining the vertices and edges of a graph G. there are two standard ways for graph traversal:

- **Breadth first search**
- **Depth first search**

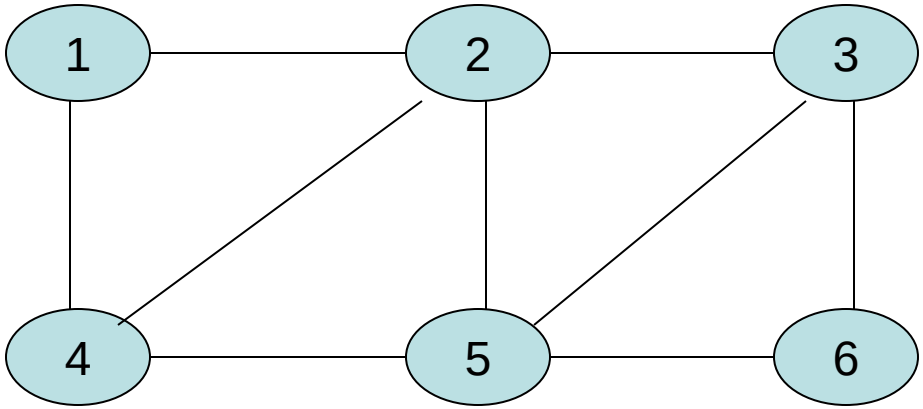
Breadth first search

- Given an input graph G=(V,E) and source vertex s, from where to begin.
- The BFS systematically explores the edges of G to discover every vertex that is reachable from s.
- It produces a breadth first tree with root s that contains all such vertices that are reachable from s.
- For every vertex v reachable from s, the path in the breadth first tree from s to v corresponds to a shortest path.

Breadth first search

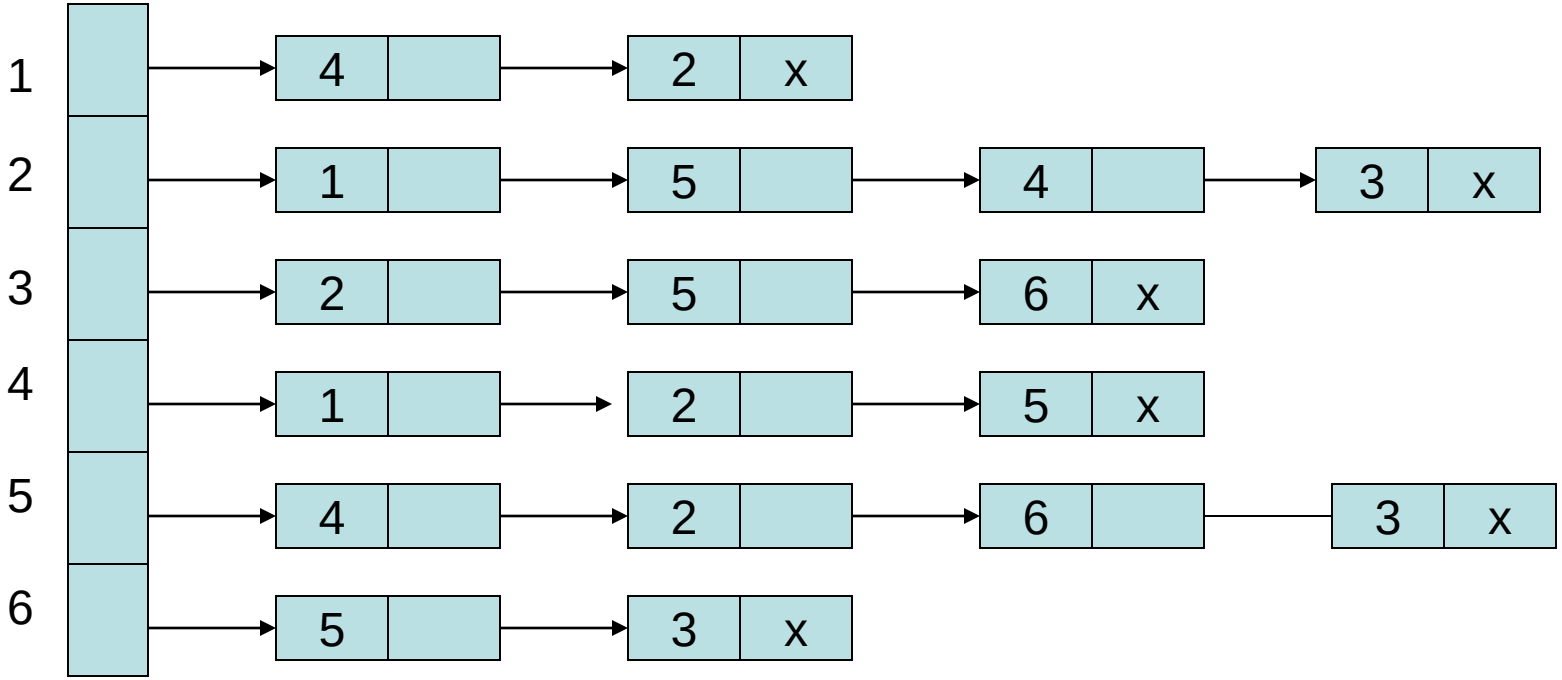
- During the execution of the algorithm, each node n of G will be one of the three states, called the status of n as follows:
- Status=1: (ready state) the initial state of the node n
- Status=2: (waiting state) the node n is on the queue or stack waiting to be processed.
- Status=3: (processed state) the node has been processed.

Example



Undirected Graph

Adjacency List Representation



Adjacency list for undirected graph

Algorithm

Step 1:Initialize all nodes to ready state (status =1)

Step 2: Put the starting node in queue and change its status to the waiting state (status=2)

Step 3: Repeat step 4 and 5 until queue is empty

Step 4: Remove the front node n of queue. Process n and change the status of n to the processed state (status=3)

Step 5: Add to the rear of the queue all the neighbor of n that are in ready state (status=1), and change their status to the waiting state (status=2)

[end of the step 3 loop]

Step 6: exit

BFS

Step 1: Initially add 2 to the queue

F=0

R=0

F						
2						
R						

Step 2:remove the front element 2 from queue by setting front=front +1 add to the queue the neighbors of 2

F=1

R=4

	F					
2	1	5	4	3		
				R		

BFS

Step 3:Remove the front element 1 from queue by setting front=front +1 add to the queue the neighbors of 1

F=2

R=4

F

2	1	5	4	3		

R

Step 4:Remove the front element 5 from queue by setting front=front +1 add to the queue the neighbors of 5

F=3

R=5

F

2	1	5	4	3	6	

R

BFS

Step 5: 4:Remove the front element 4 from queue by setting front=front +1 add to the queue the neighbors of 4

F=4

R=5

F

2	1	5	4	3	6	

R

Step 6:Remove the front element 6 from queue by setting front=front +1 add to the queue the neighbors of 6

F=5

R=5

F

2	1	5	4	3	6	

R

Depth First Search

- The DFS, as the name implies, is to search deeper in the graph, whenever possible.
- The edges are explored out of the most recently discovered vertex v that still has unexplored edges leaving it.
- When all of v's edges have been explored, the search backtracks to explore edges leaving the vertex from which v was discovered.
- This process continue until we have discovered all the vertices that are reachable from the source vertex.
- DFS uses stack to maintain the order in which the vertices are to be processed.

Algorithm

Step 1:Initialize all nodes to ready state (status =1)

Step 2: Push the starting node in stack and change its status to the waiting state (status=2)

Step 3: Repeat step 4 and 5 until stack is empty

Step 4: pop the top node n of stack. Process n and change the status of n to the processed state (status=3)

Step 5: Push on to stack all the neighbor of n that are in ready state (status=1), and change their status to the waiting state (status=2)

[end of the step 3 loop]

Step 6: exit

DFS

Step1: Initially, push 3 on to the stack as follows:

stack: 3

Step2: pop and print the top element 3 and push onto the stack all the neighbors of 3 (those are in the ready state) as follows:

print:3 stack: 2, 5, 6

Step3: pop and print the top element 6 and push onto the stack all the neighbors of 6 (those are in the ready state) as follows:

print:6 stack: 2, 5

DFS

Step4: pop and print the top element 5 and push onto the stack all the neighbors of 5 (those are in the ready state) as follows:

print:5 stack: 2,4

Step5: pop and print the top element 4 and push onto the stack all the neighbors of 4 (those are in the ready state) as follows:

print:4 stack: 2, 1

DFS

Step6: pop and print the top element 1 and push onto the stack all the neighbors of 1 (those are in the ready state) as follows:

print:1 stack: 2

Step7: pop and print the top element 2 and push onto the stack all the neighbors of 2 (those are in the ready state) as follows:

print:2 stack:

Now the sequence is 3,6,5,4,1,2

Applications of Graphs

Minimum spanning Tree:

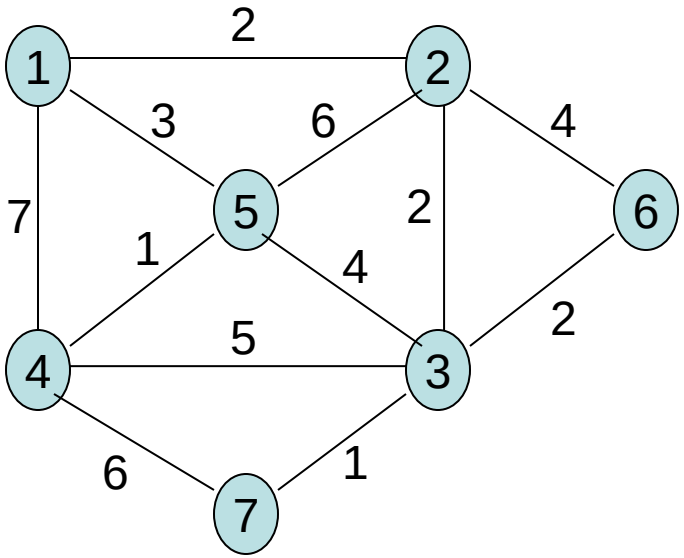
- Spanning tree for a graph G=(V,E), is a subgraph of a G that is tree and contains all the vertices of G.
- In a weighted graph, the weight of a graph is the sum of the weights of the edges of the graph.
- A minimum spanning tree for a weighted graph is a spanning tree with minimum weight
- If Graph G with n vertices, then the MST will have n-1 edges, assuming that the graph is connected.
- In general, a weighted graph may have more than one MST.
- If G is not connected, then it can not have any spanning tree. In this case it will have spanning forest.

MST

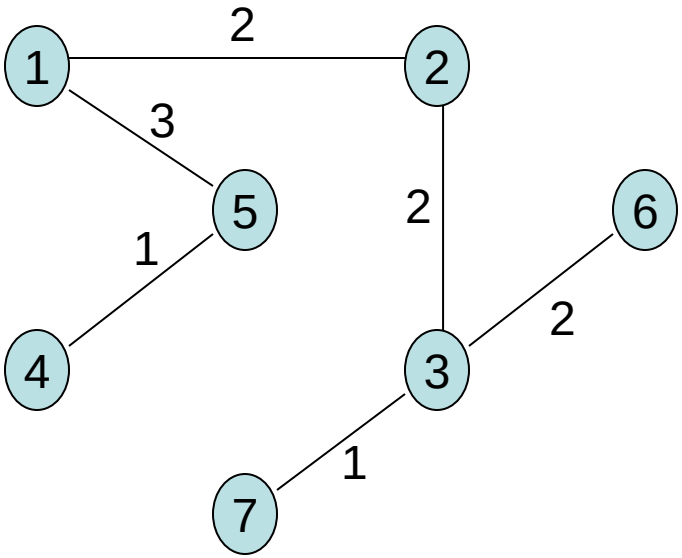
Situation where MST must be found:

- We want to find cheapest way to connect a set of terminals, where terminals may represent cities, electrical/electronic components of a circuit, computers or factories, by using say roads, wires, or telephone lines. The solution to this is MST, which has an edge for each possible connection weighted by the cost of that connection.
- It is also an important sub problem in various routing algorithms. By routing means finding efficient paths through a graph that visit every vertex.

MST



Given Weighted Graph



Minimum spanning tree

MST

- Prim Algorithm:

It begins by selecting an arbitrary starting vertex, and then branches out from the part of the tree constructed so far by choosing a new vertex and edge at each iteration.

During the course of the algorithm, the vertices may be thought of as divided into three disjoint categories as follows:

- **Tree vertices-** those in the tree constructed so far.
- **Fringe vertices-** those vertices that are not in tree, but are adjacent to some vertex in the tree
- **Unseen vertex-** remaining vertices of the graph

Prim Algorithm

- The key step in the algorithm is the selection of a vertex from the fringe vertices and an incident edge. Actually, since the weights are on the edges, the focus of the choice is on the edge, not the vertex.
- Prim’s algorithm always chooses an edge from a tree vertex to fringe vertex of minimum weight.

Prim Algorithm

Begin

Select an arbitrary vertex to start the tree

while there are fringe vertex **do**

Select an edge of minimum weight between a tree and a fringe vertex

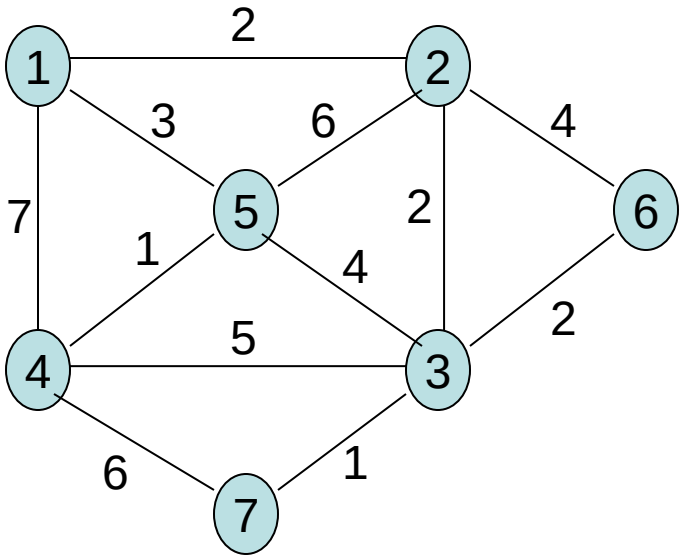
Add the selected edge and the fringe vertex to the tree

endwhile

End

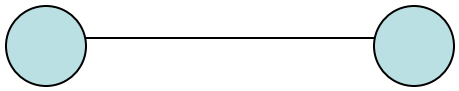
Prim Algorithm

- After each iteration of the algorithm loop, there may be new fringe vertices, and the set of edges from which the next selection is made. For each fringe vertex, we need to keep track of only one edge to it from the tree– the one of the lowest weight. We will call such edges candidate edges.



1		2	2		5	3		4	7	x				
2		5	6		6	4		3	2		1	2	x	
3		5	4		4	5		2	2		6	2		7	1	x
4		1	7		5	1		3	5		7	6	x	
5		1	3		2	6		3	4		4	1	x	
6		2	4		3	2	x							
7		4	6		3	1	x							

Following notations for tree edges and fringe edges will be used

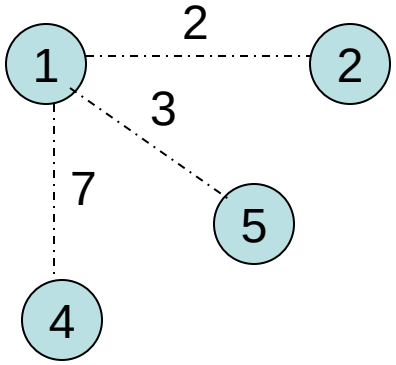


Tree edge

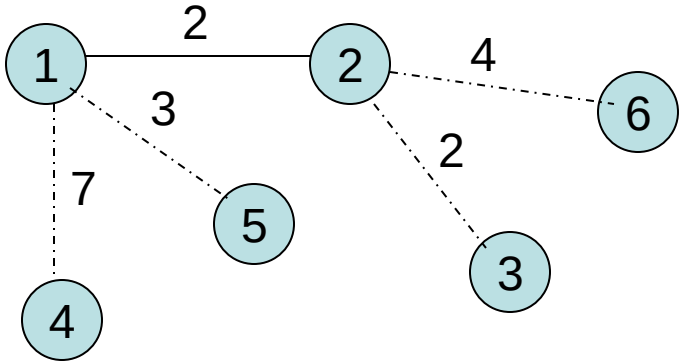
Fringe edge

The procedure start with vertex 1 as progress as shown below.

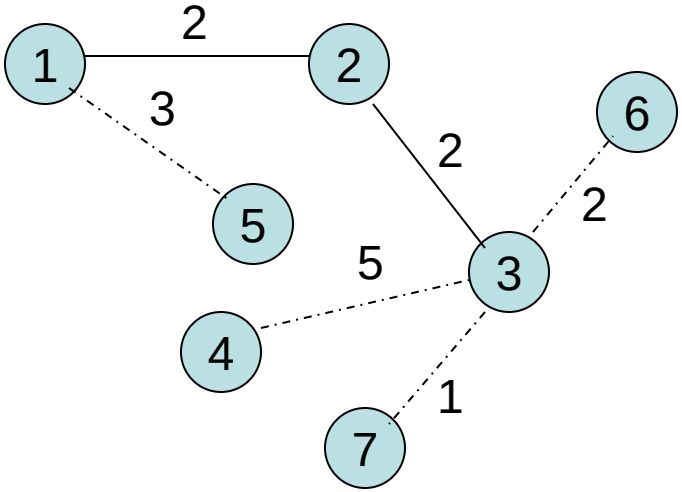
(a)



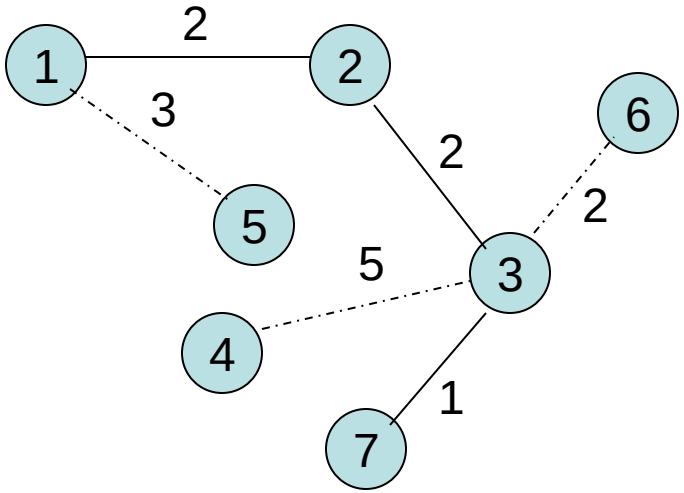
(b)



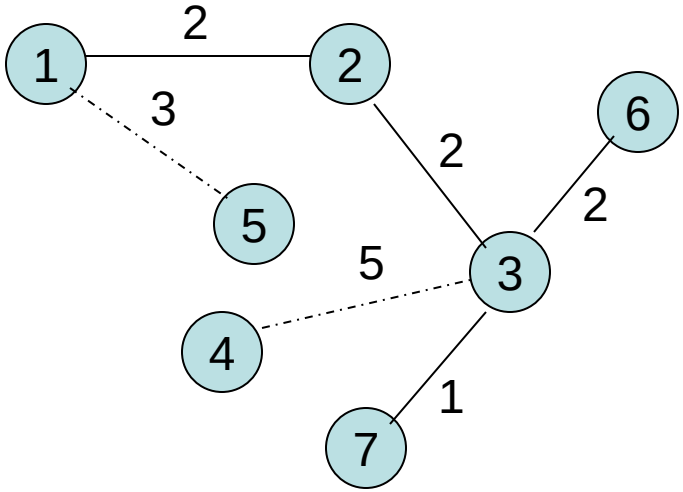
(c)


1
2
3
4
5
6
7
2
3
2
2
5
1

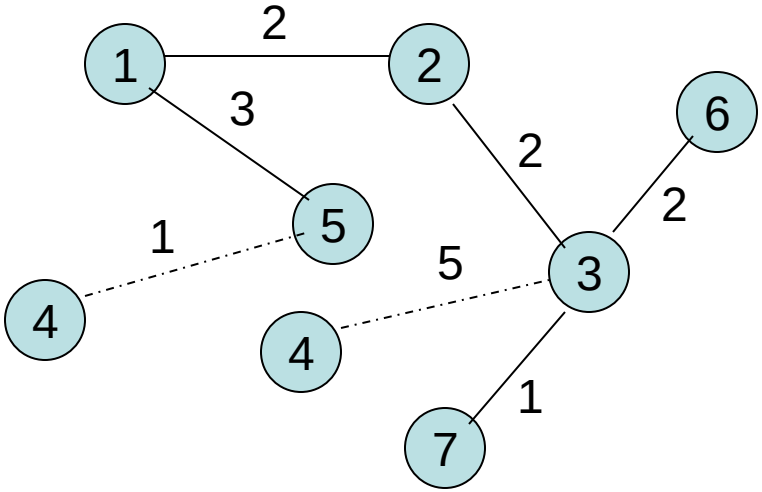
(d)


1
2
3
4
5
6
7
2
3
2
2
5
1

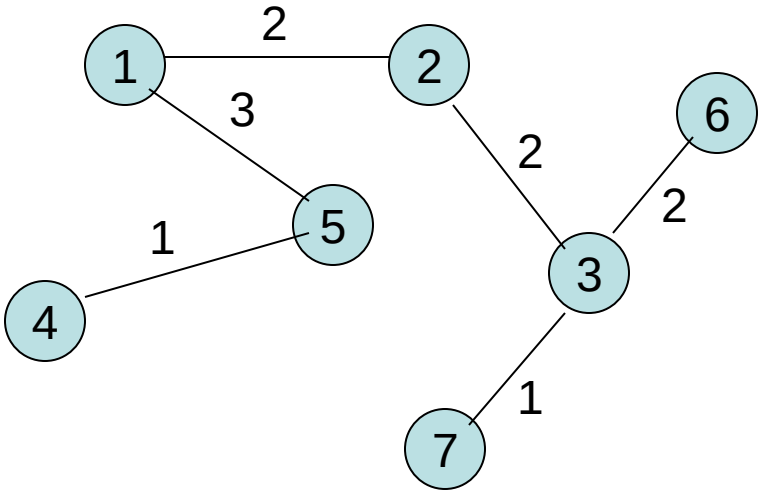
(e)


1
2
3
4
5
6
7
2
3
2
2
5
1

(f)


1
2
3
4
5
6
7
2
3
2
2
1
5
1

(g)



(h)

(a)—(h) illustrates progress of MST algorithm

Topological Sort

A topological sort of a directed graph without cycles, also known as directed acyclic graph or simple DAG, G=(V,E) is a linear ordering of all its vertices such that if G contains an edge (u,v), then u appears before v in the ordering.

Note: if the graph contains cycle(s), i.e, graph is not DAG, then no linear ordering is possible.

A topological sort of a graph can be viewed as an ordering of its vertices along a horizontal line so that all directed edges go from left to right.

Directed acyclic graphs are used in many applications to indicate precedence among events.

Topological sort

To find topological sort of a DAG, the general structure of the algorithm can be described as follows:

Begin

Perform DFS on G for each vertex

As each vertex is finished, insert at front of a linked list

Print the elements of linked list in order

End

To find topological sort of a DAG, the procedure to perform DFS visit to vertex is slightly modified as given in the algorithm, where instead of printing the vertex, it is inserted in the beginning of the linked list.

DfsVisitModified(adj, n, u)

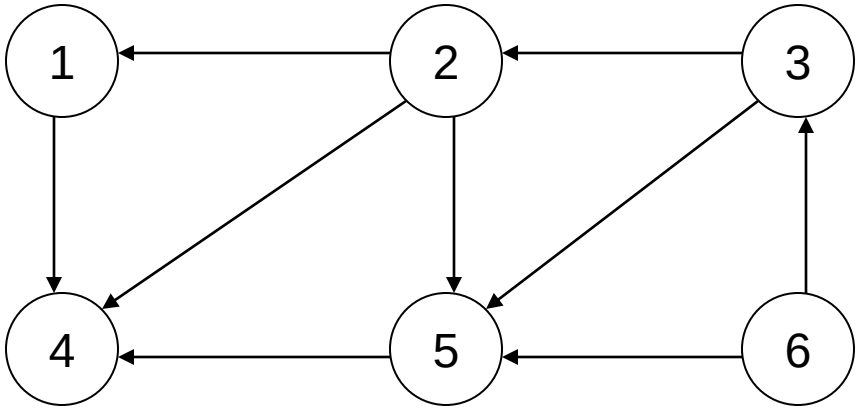
Here adj is the adjacency list of the graph with n vertices, and vertex u is the vertex to be visited. This algorithm uses recursion.

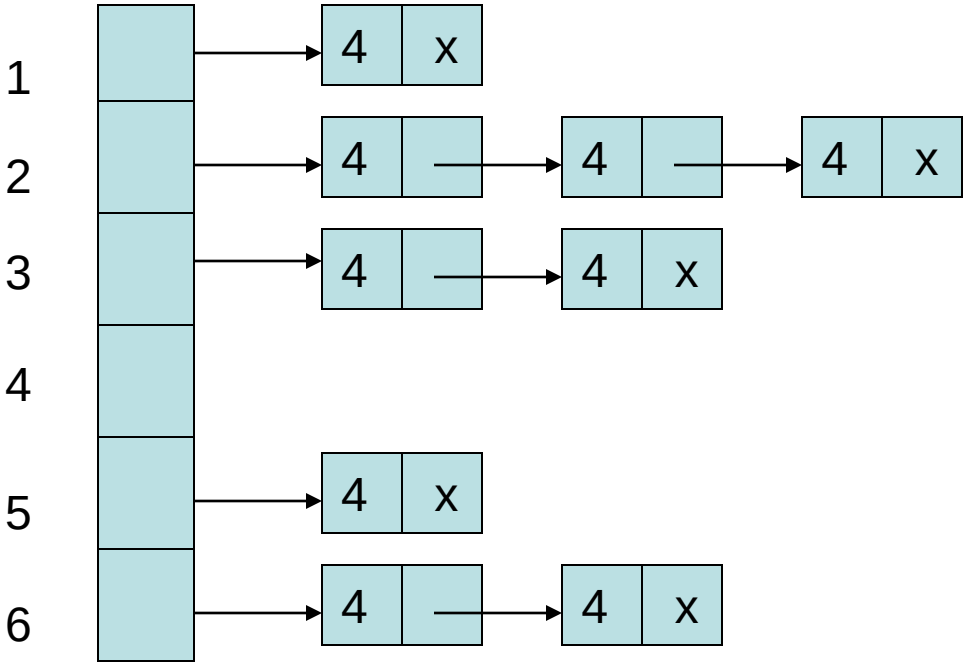
```
Begin
    set color[u]=GRAY
    set ptr= adj[u]
    while (ptr!=NULL) do
        set v=ptr->info
        if (color[v]=WHITE) then
            call DfsVisitModified(adj, n, v)
        endif
        set ptr=ptr->next
    endwhile
    insert vertex u in beginning of linear linked list LIST
    set color[u]=BLACK
End
```

TopologicalSort(adj, n)

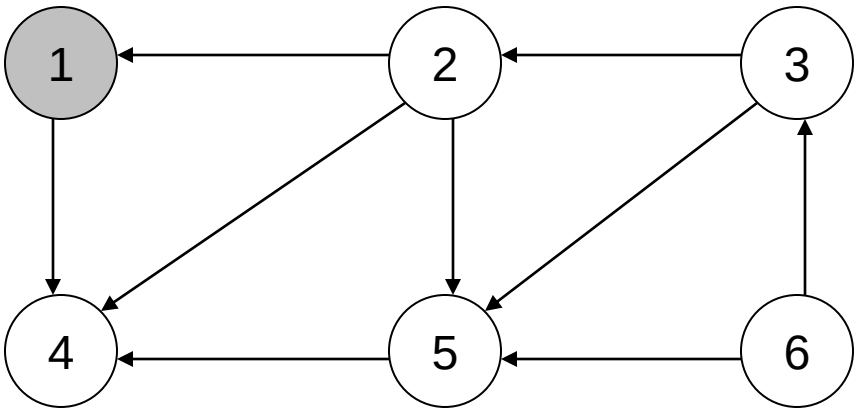
Here adj is the adjacency list of the graph with n vertices. This algorithm finds the topological sort of the graph G. It is linear linked list LIST to store the visited by DFS visit procedure in order of their traversal.

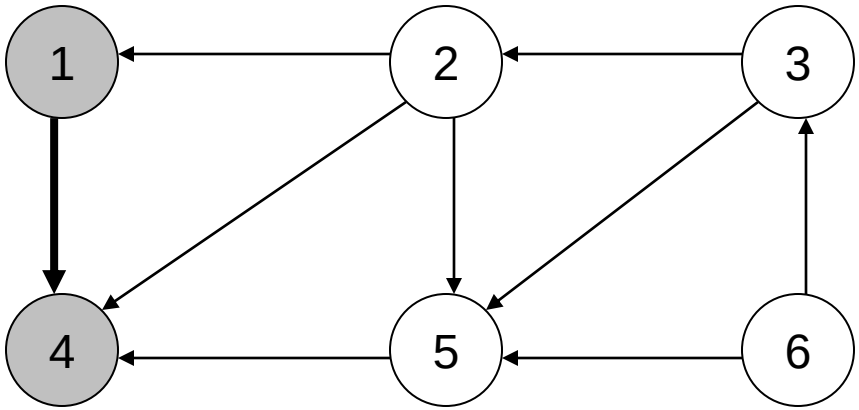
```
Begin
    create linear linked list LIST
    for i=1 to n by 1 do
            set color[i]=WHITE
    endfor
    for i=1 to n by 1 do
            if(color[i]=WHITE) then
                        call DfsVisitModified(adj, n, i)
            endif
    endfor
    set ptr=LIST
    while( ptr!=NULL) do
            print ptr->info
            set ptr=ptr->next
    endwhile
end
```

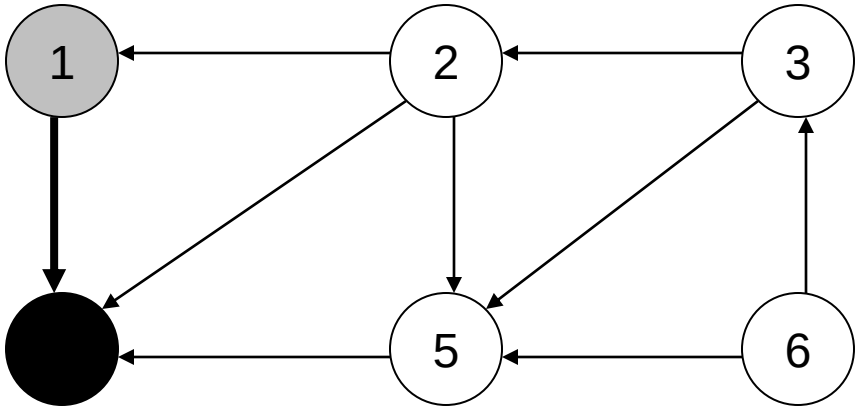


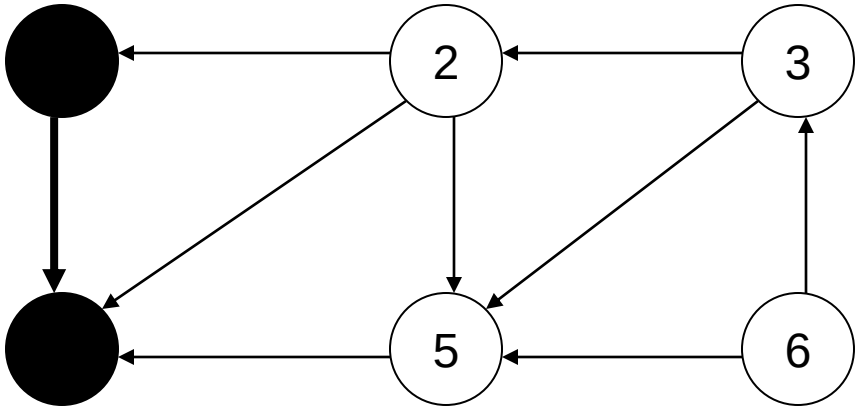


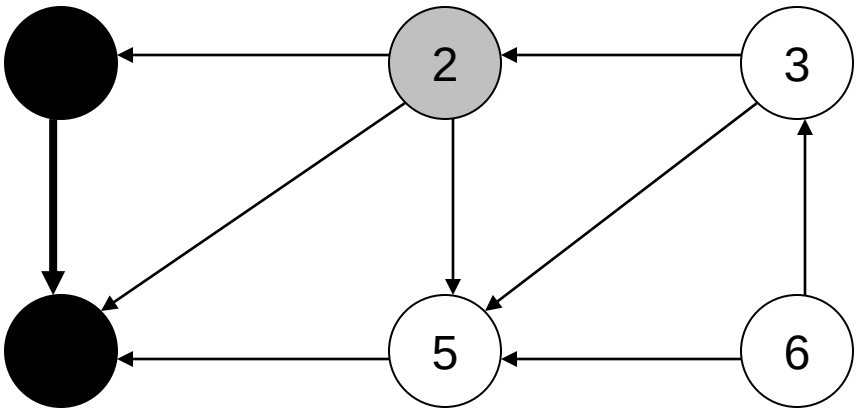
Directed acyclic graph and its adjacency list

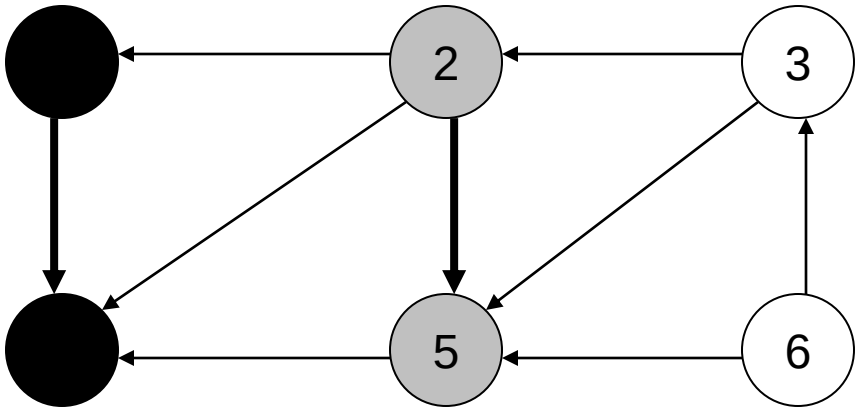

1
2
3
4
5
6

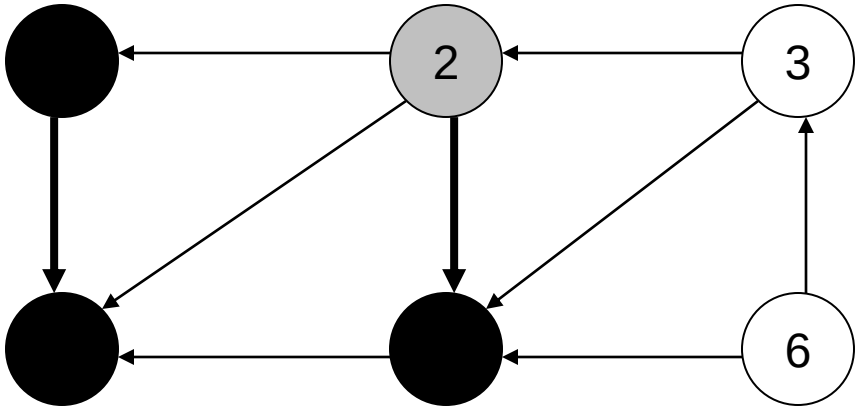

1
2
3
4
5
6

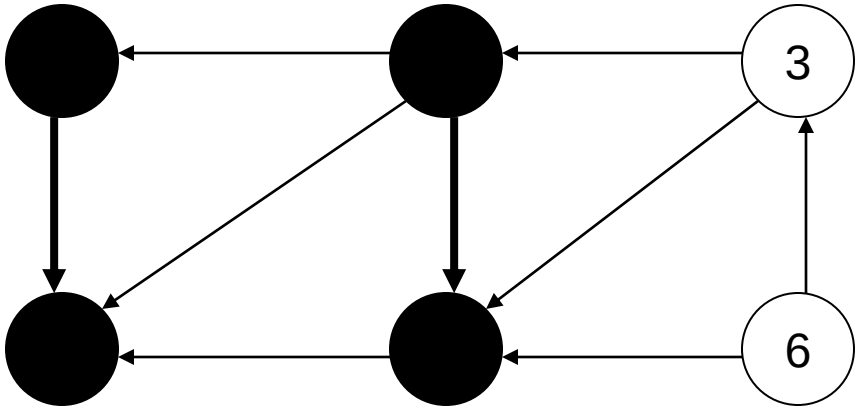

1
2
3
5
6

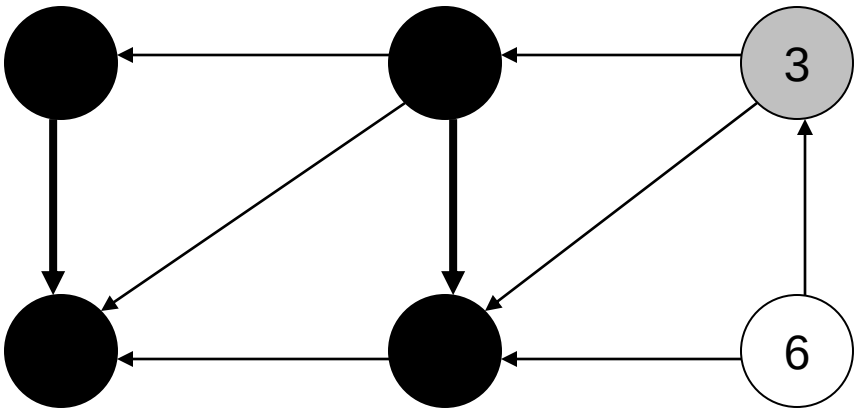

2
3
5
6

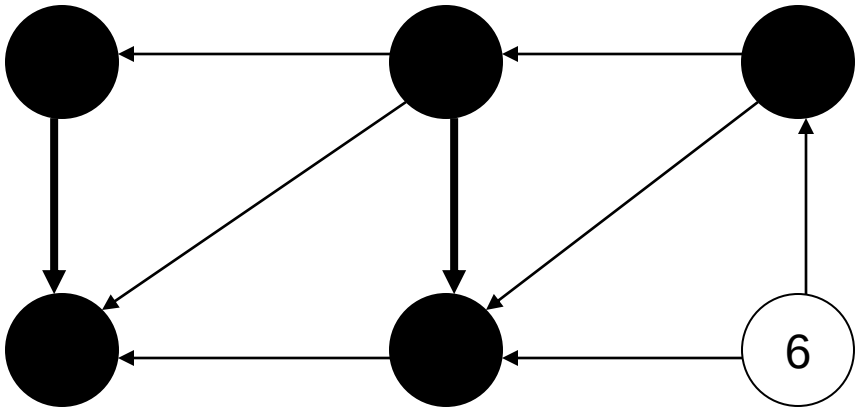

2
3
5
6

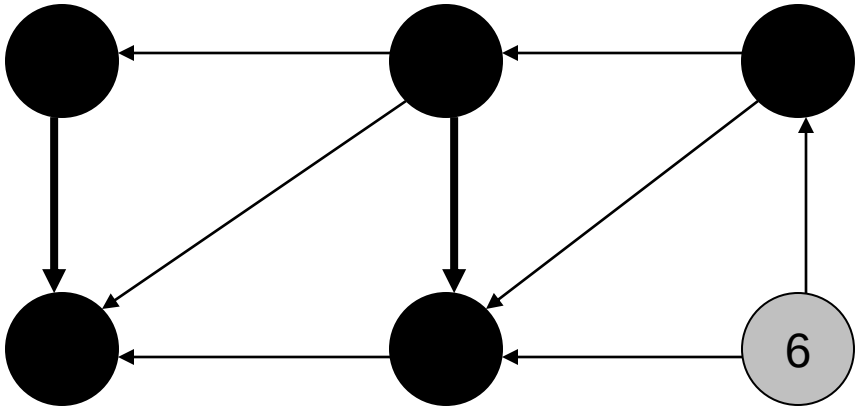

2
3
5
6

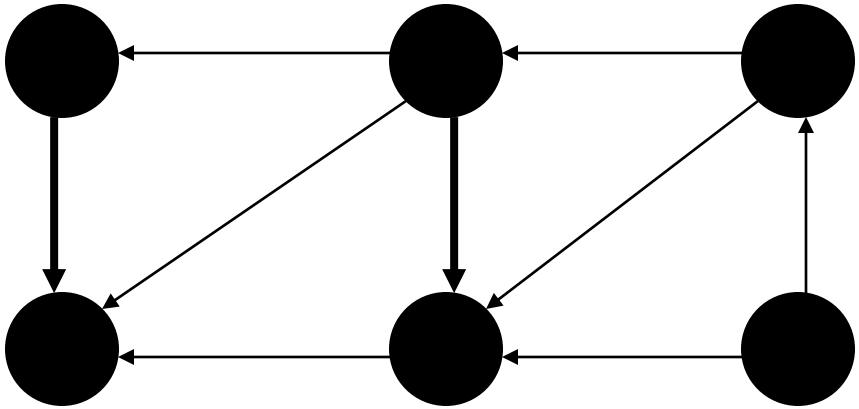

2
3
6

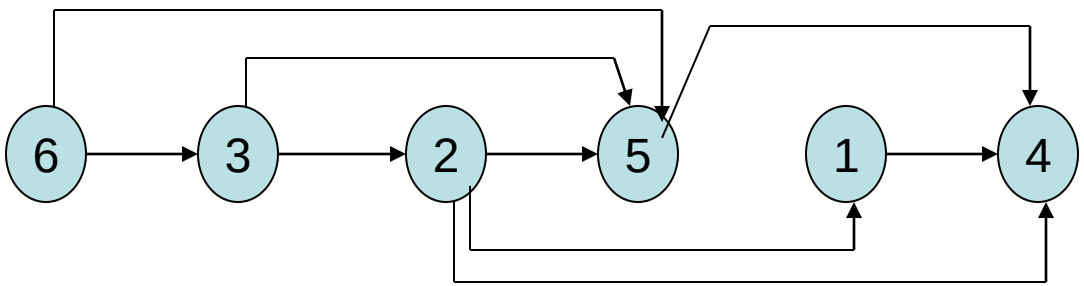

3
6


3
6


6


6





Topological sort

Dijkstra’s Algorithms

- Let G be a weighted graph with n vertices V1,V2..Vn.
- Suppose G=(V,E,We) is weighted graph. i.e each edge e in G is assigned a non negative number, we called the weight or length of the edge e.
- Consider a starting vertex. Dijkstra’s algorithm will find the weight or length to each vertex from the source vertex.

Dijkstra’s Algorithms

- Set V=(V1,V2…Vn) contains the vertices and edges E=(e1,e2…em) of the graph G.
- W(e) is the weight of an edge e. which contains the vertex v1 and v2.
- Q is the set of vertices, which are not visited.
- m is the vertex in Q for which weight W(m) is minimum i.e. minimum cost edge.
- S is a source vertex.

Dijkstra's Algorithms

1. Input the source vertex and assign it to S.

   (a) set W(s)=0

   (b) set W(v)=_____for all vertices V is not equal to S.

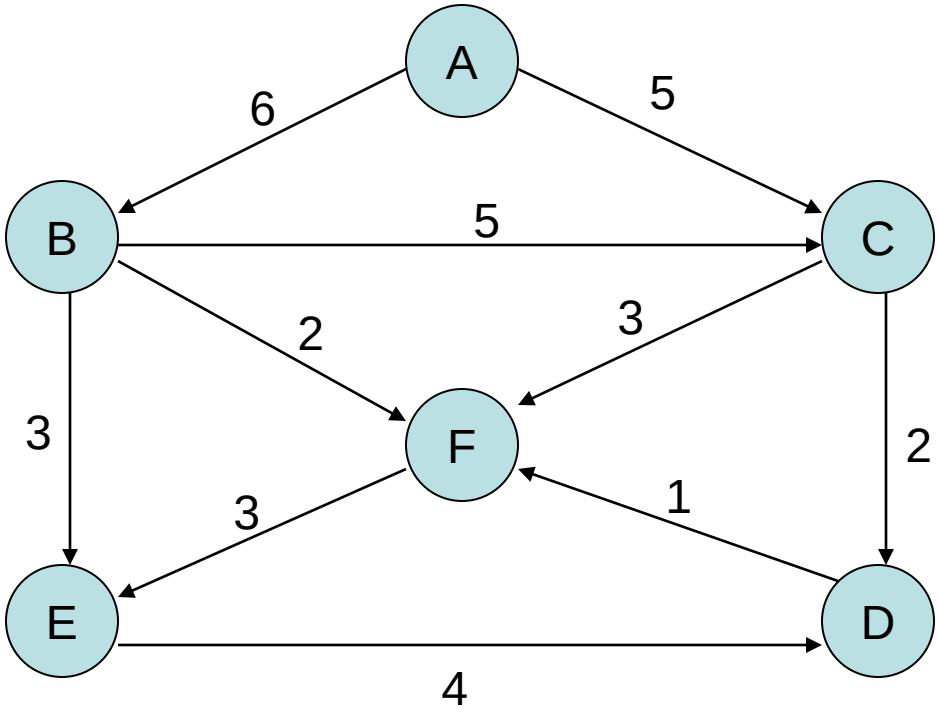
2. Set Q=V which is a set of vertices in the graph.
3. Suppose m be the vertices in Q for which W(m) is minimum.
4. Make the vertices m as visited and delete it from the set Q.
5. Find the vertices I which are incident with m and member of Q (that is vertices which are not visited)
6. Update the weight of vertices I={i1,i2…ik) by

   (a) W(i1)=min[W(i1),W(m)+W(m,i1)]

Dijkstra’s Algorithms

7. If any change is made in W(v), store the vertices to corresponding vertices I, using the array, for tracing the shortest path.
8. Repeat the process from step 3 to 7 until the set Q is empty.
9. Exit

Dijkstra’s Algorithms



Dijkstra's Algorithms

Source vertex=A

W(A)=0

V={A,B,C,D,E,F} =Q

V	A	B	C	D	E	F
W(V)	0					
Q	A	B	C	D	E	F

A	
B	
C	
D	
E	
F	

Dijkstra's Algorithms

ITERATION 1:

m=A

W(A,A)=0

Q=={B,C,D,E,F}

I={B,C}…incident edges

W(B)=min[W(B),W(A)+W(A,B)]

=min(_,0+6)

=6

W(C)=min[W(C),W(A)+W(A,C)]

=min(_,0+5)

=5

V	A	B	C	D	E	F
W(V)	0	6	5			
Q		B	C	D	E	F

A	
B	A
C	A
D	
E	
F	

Dijkstra's Algorithms

ITERATION 2:

m=C (because W(v) in min vertex and is also a member of Q)

Q={B,D,E,F}

I={D,F}…incident edges

W(D)=min[W(D),W(C)+W(C,D)]

=min(_,[5+2])

=7

W(F)=min[W(F),W(C)+W(C,F)]

=min(_,[5+3])

=8

V	A	B	C	D	E	F
W(V)	0	6	5	7		8
Q		B		D	E	F

A	
B	A
C	A
D	C
E	
F	C

Dijkstra's Algorithms

ITERATION 3:

m=B (because W(v) in min vertex and is also a member of Q)

Q={D,E,F}

I={E,F}…incident edges….(C IS NOT A MEMBER OF Q)

W(E) =min(_,[6+3])

=9

W(F)=min(_,[6+2])

=8

V	A	B	C	D	E	F
W(V)	0	6	5	7	9	8
Q				D	E	F

A	
B	A
C	A
D	C
E	B
F	C

Dijkstra's Algorithms

ITERATION 4:

m=D (because W(v) in min vertex and is also a member of Q)

Q={E,F}

I={F}…incident edges

W(F)=min(8,[7+1]])

=8

V	A	B	C	D	E	F
W(V)	0	6	5	7	9	8
Q					E	F

A	
B	A
C	A
D	C
E	B
F	C

Dijkstra's Algorithms

ITERATION 5:

m=F (because W(v) in min vertex and is also a member of Q)

Q={E}

I={E}…incident edges….

W(E)=min(9,[8+3]])

=9

V	A	B	C	D	E	F
W(V)	0	6	5	7	9	8
Q					E	

A	
B	A
C	A
D	C
E	B
F	C

Dijkstra's Algorithms

Now E is only chain, hence we stop the iteration and the final table is :

If the source vertex is A and destination vertex is D then the weight is 7 and the shortest path can be traced from table at right side.

V	A	B	C	D	E	F
W(V)	0	6	5	7	9	8

A	
B	A
C	A
D	C
E	B
F	C